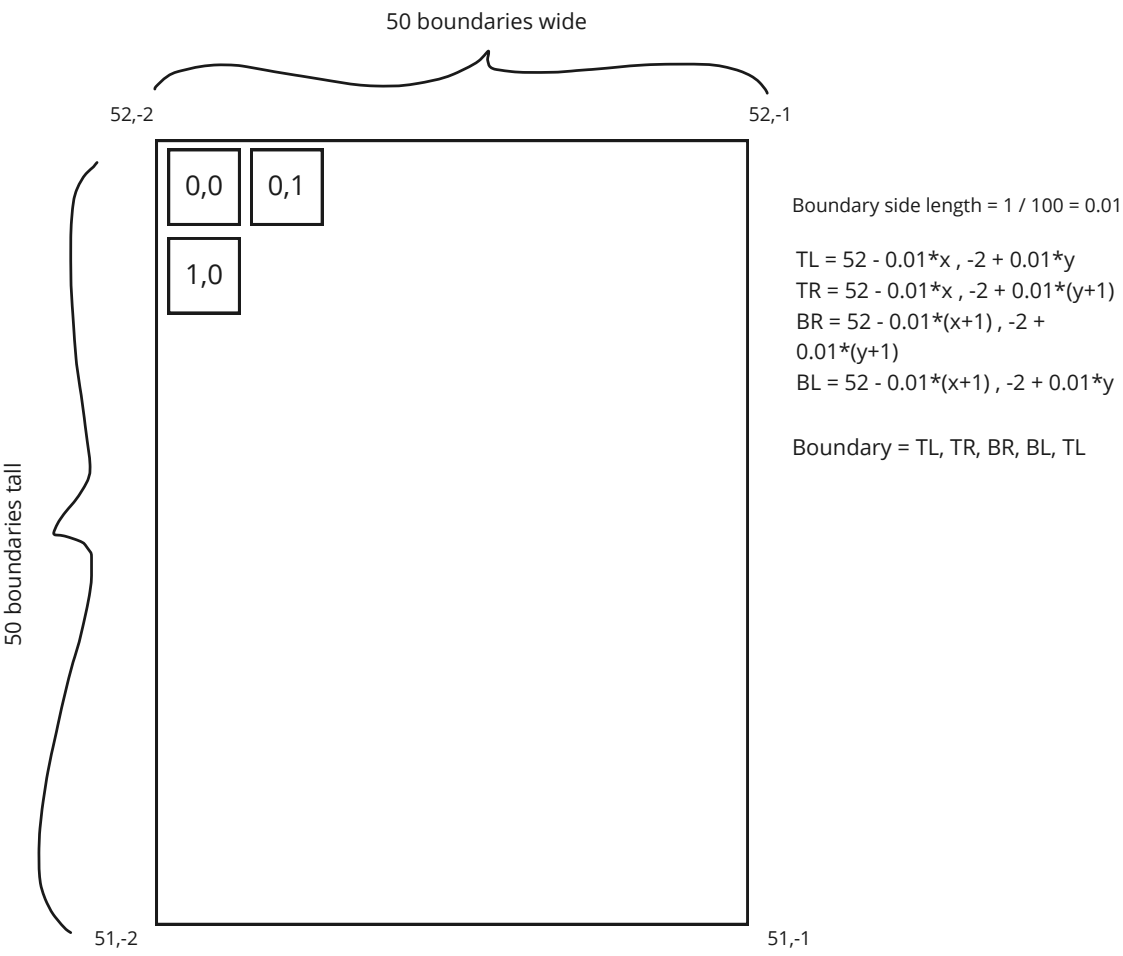




Boundary side length = $1 / 100 = 0.01$

$TL = 52 - 0.01 * x, -2 + 0.01 * y$

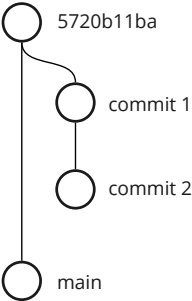
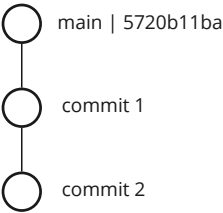
$TR = 52 - 0.01 * x, -2 + 0.01 * (y+1)$

$BR = 52 - 0.01 * (x+1), -2 + 0.01 * (y+1)$

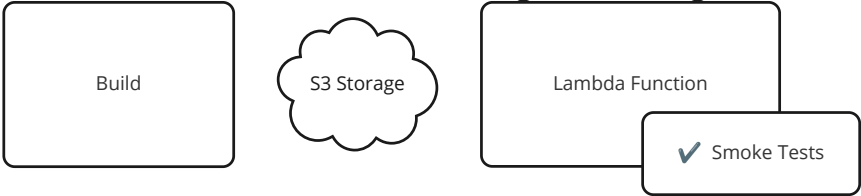
$BL = 52 - 0.01 * (x+1), -2 + 0.01 * y$

Boundary = TL, TR, BR, BL, TL

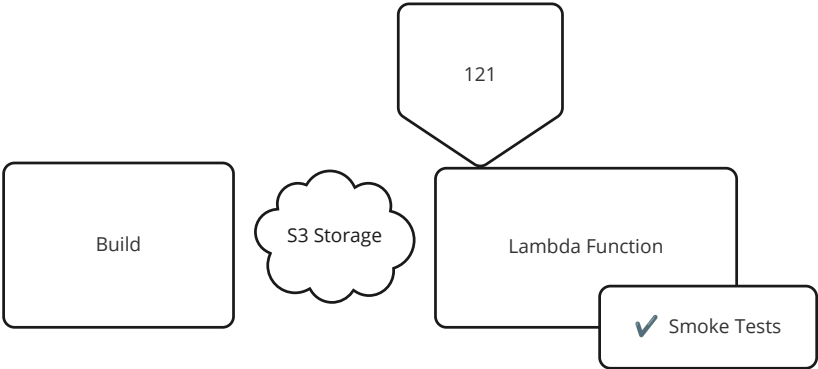
○ main | 5720b11ba



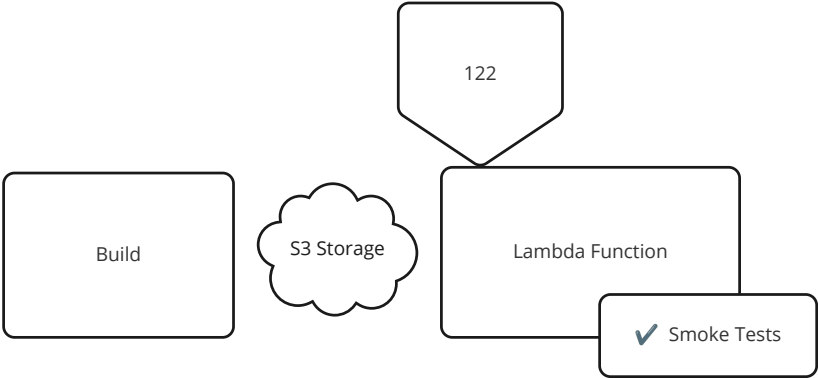
backend/services/flex
code is merged to main



backend/services/flex
code is merged to main

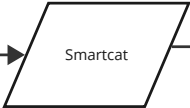
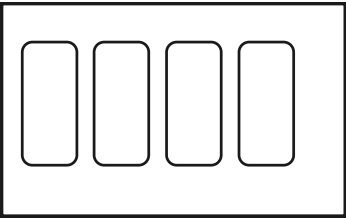


backend/services/flex
code is merged to main

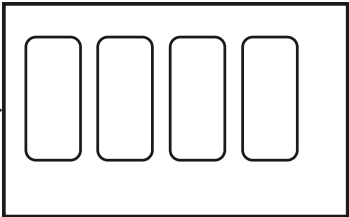


Original Approach

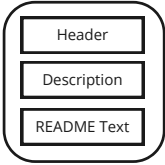
Production Templates

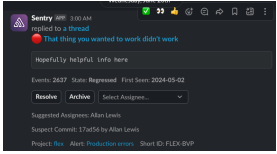


Smart Cat Translated Folder



End Goal





Did we send the number of messages we expected during that event?

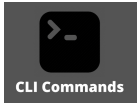
What messages have been sent to my service in the last 3 hours?



Slow(ish) to load

Interacting with data ends in the UI

Partial results over large queries



Slow to formulate the right command

Not a particularly good user experience

I can pipe my output into anything I want



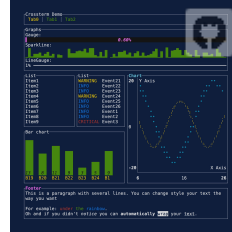
curses - 1978



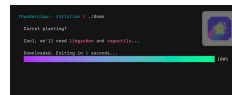
ncurses - 1993



ratatui (Rust)

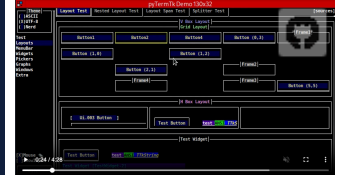


bubbletea (Go)

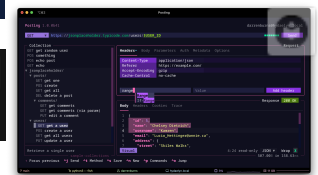


2024

pyTermTk (Python)



textual (Python)



Which orgs used the X API endpoint in the past 30 minutes

Lines of code in each backend service

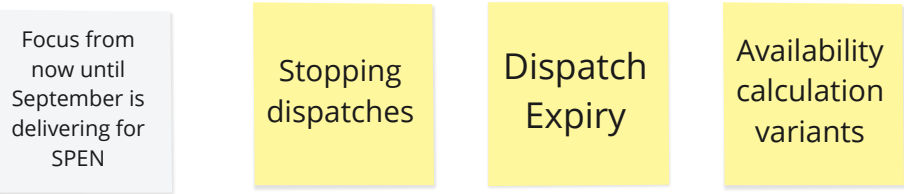
68605 flex
49389 files
27119 metering
21228 bids
9559 dps
8665 integrationnnged
8619 contracts
7325 dispatch
7096 integrationnngeso
5532 emails
4458 upload
3432 authservice
2997 exchange
2748 webhook
2023 registration
1872 dfssettlement
1705 antivirus
1390 area
999 analytics
985 integrationnonenet
934 markets
907 mockdata
898 qualification
701 features
633 milestones
480 maxdashboard

50% of bids = 10500

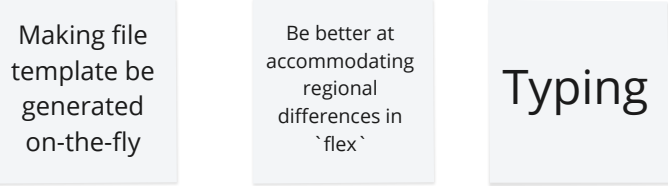
50159

134299

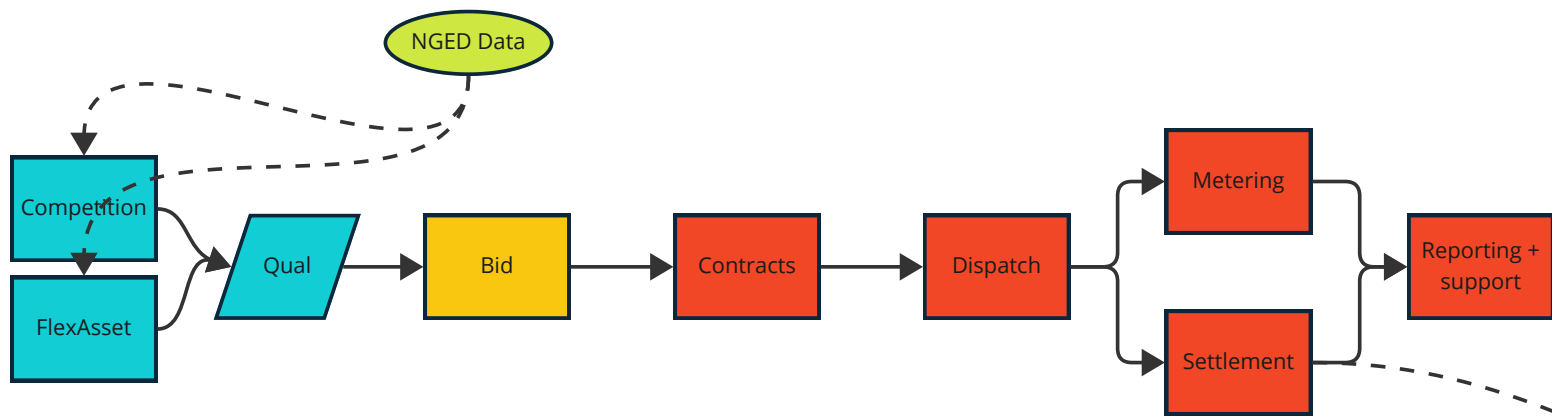
Market Outcome Priorities



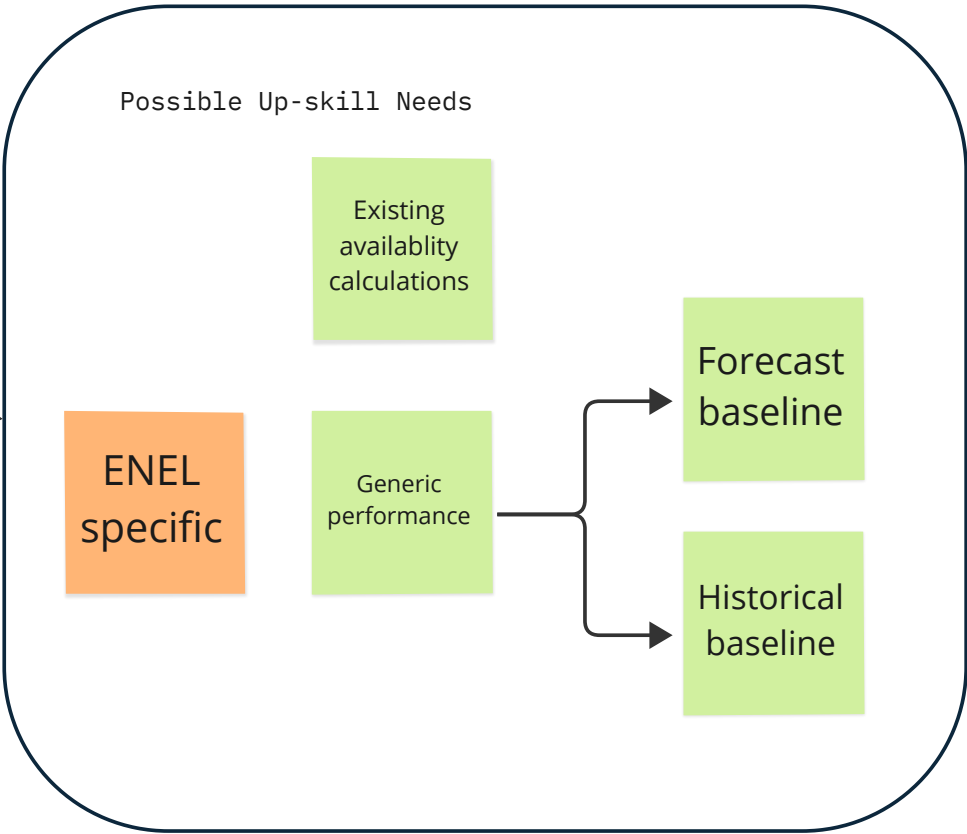
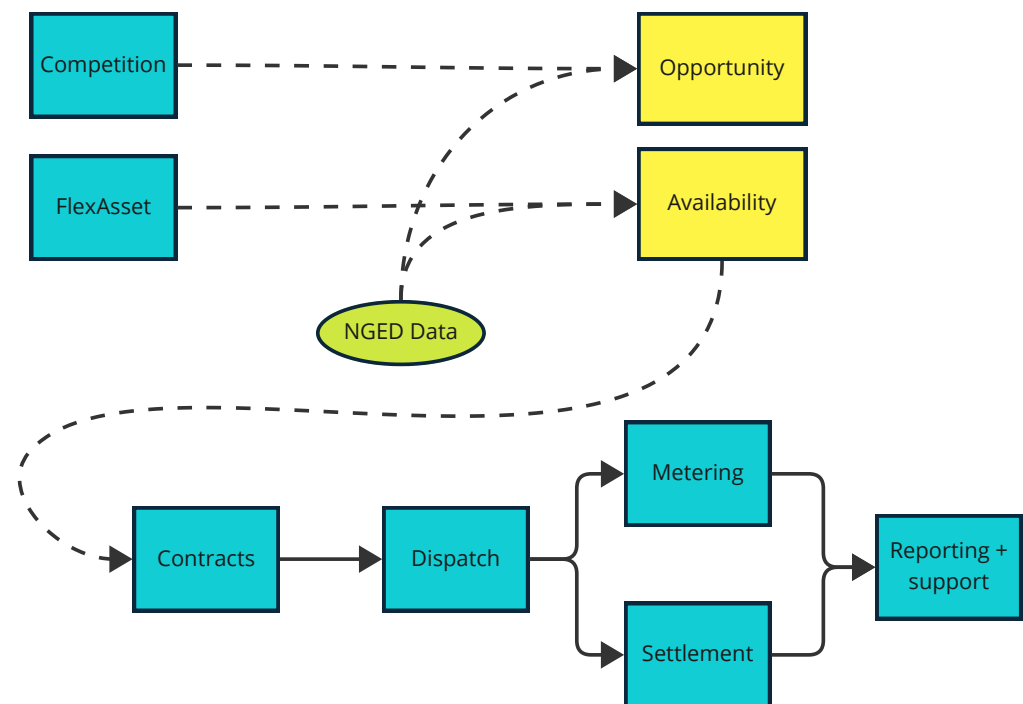
Market Structure Priorities



Market Structure Market Access Market Participation Market Outcomes



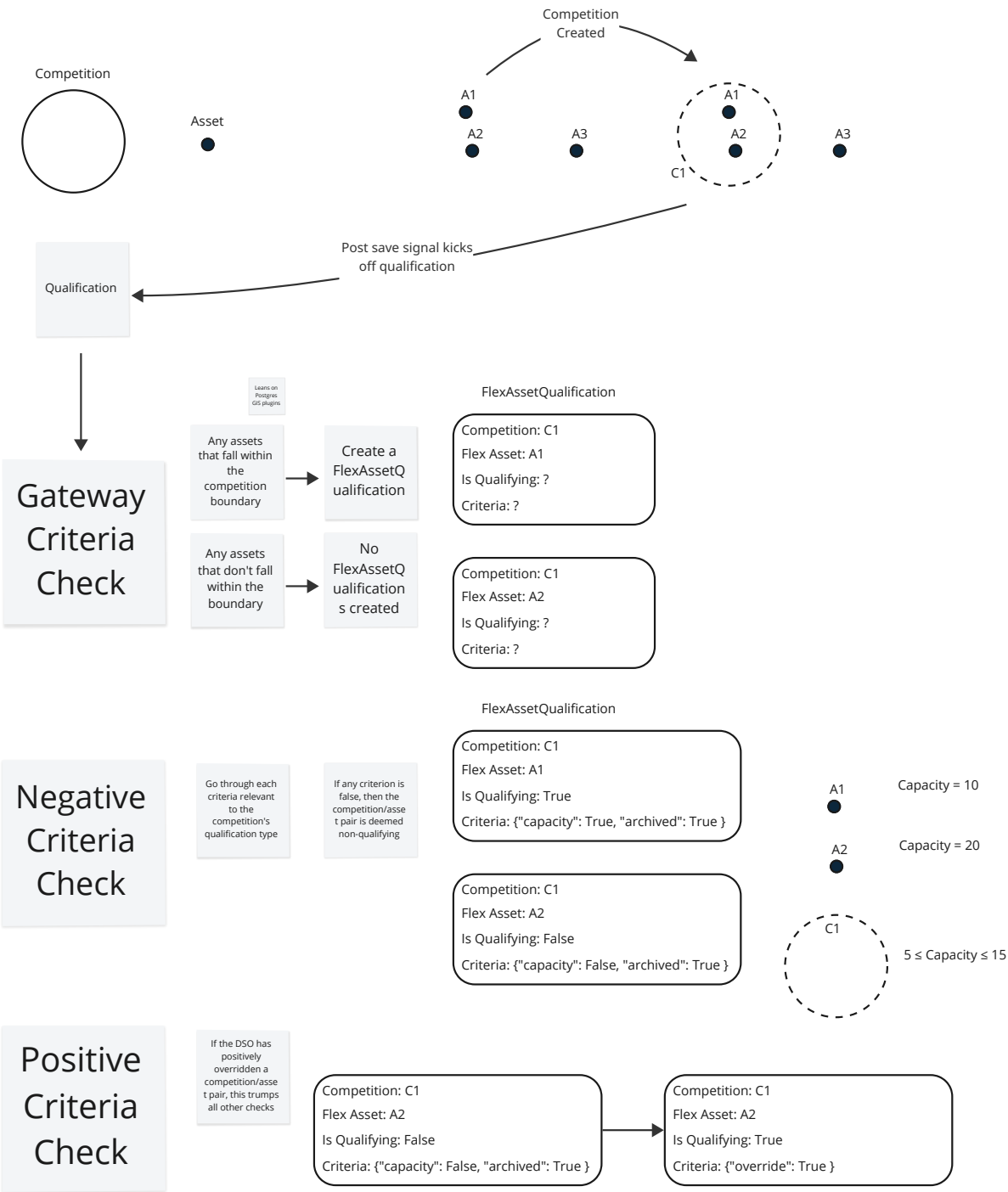
Flex Gateway Market Integrations Marketplace



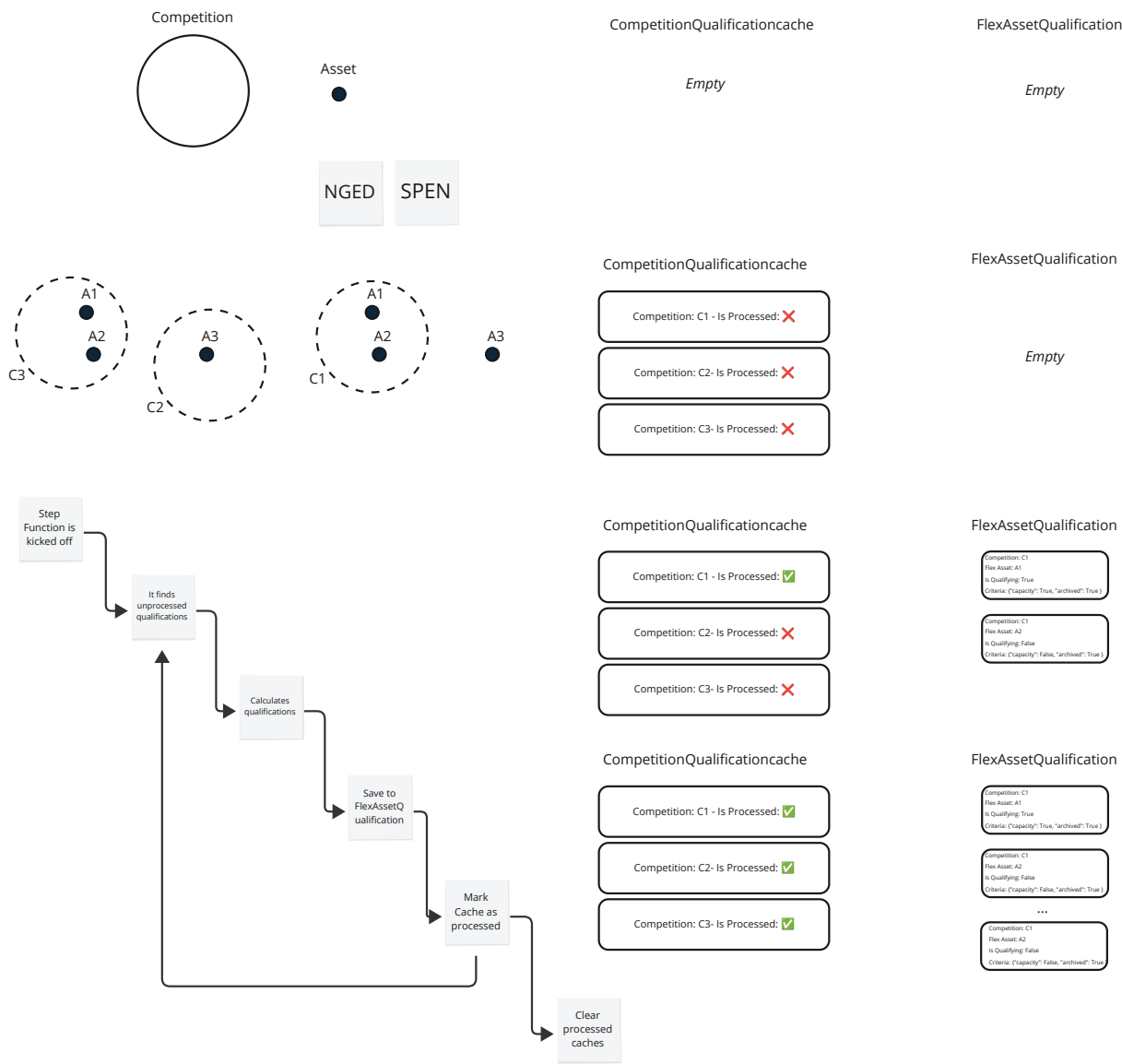
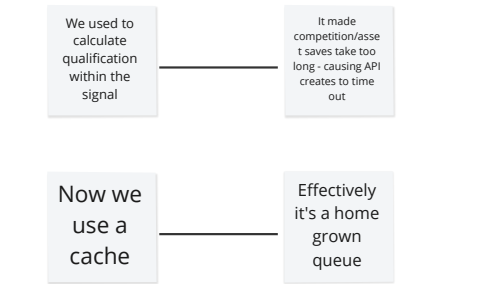
How I feel about particular concepts

Concepts / Threads	Comfortable with this	Open threads and thoughts	Thoughts from Adrian	Action			
Cached qualification	Supporting cached qualification runs	The split between cached and non-cached. Are non-cached ever going to be run in production going forward?	Why aren't they getting cleared				
Postcode Area data	You've regularly shared thinking on how to improve this	The wide choice of options. I suppose something is better than what we currently have.	Didn't appreciate how dynamic postcode data is	Voronoi transform has to be done wholesale - you can't do partial updates	DSOs get gaps in competition zones because they don't have an exhaustive list of postcodes	Majority of zones are usually tessellating shapes. Convex hull can sometimes reduce number of gaps / islands	Can't shouldn't rely on postcodes
Update to qualification writing mechanism PR	I paused help with this because of time constraints and team changes	I'd like to revisit this once I'm confident we're good at monitoring qualification health	Attempted to bulk_create with update_conflicts but came across issues. Might be worth revisiting this	Thought about moving form python to pure SQL to update criteria results	Italy (Marco and Carolina) use positive overrides a lot to maintain a particular qualification state.	The crux of this optimisation is INSERT INTO SELECT - Which django doesn't support	
Moving asset topology checks to a gateway criteria	Understand that this could reduce strain on DB to do gateway criterias	Unsure how this aligns with market. Are most DSOs just interested in connected assets and not necessarily colocated assets	DSOs appear to be quite happy with using geography as an initial gateway check.	We could move forward to migrate to a gateway check but DSOs may need a mindset shift to accommodate			
Criteria object is created a sub-optimally	Not sure if checking existing criteria instead of just overwriting is relevant still.	This makes it difficult to do bulk creates		High Big-O complexity in order to check criteria, resulting in high memory usage			
Move towards further batching techniques	Gone from lack of memory to lack of time instead						
Buffered boundaries	Relatively happy to bin this if there aren't major reasons not to	Dwithin isn't much worse than intersects (providing you're using geometry as opposed to geography)	Probably can bin this if we conclude it's a red herring				
Looker Studio vs Tableau	Path of least resistance for me has been looker studio so I'm largely investing in that	Is that the right approach or a distraction	Looker studio suits will be more accessible	Requires a lot of work to maintain reporting library	Tableau owner: Fee in the interim	Look studio access: Jack Rowntree	
Tools to optimise things	Running ANALYSE / EXPLAIN in DBeaver?	Running ANALYSE on the customer analytics database doesn't have the same indexing as production databases - so may not be the best way to do things	Use this for validation of data as opposed to optimisations	EC2 Admin is better	Try to get access again to EC2 and DBeaver	Make sure I can access ANALYSE / EXPLAIN on QA database	
WG-AI	Talk to Andy about ChatGPT licenses						

What is a qualification

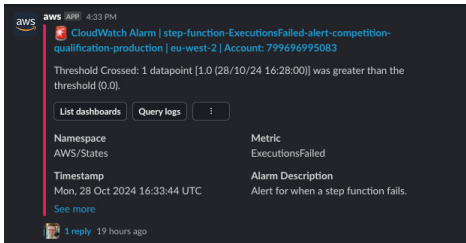


What is a qualification cache?



Why Now?

Clear processed caches

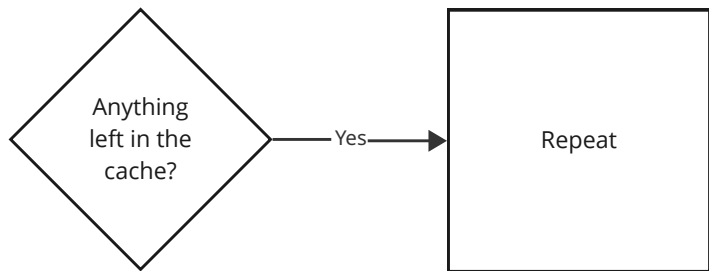
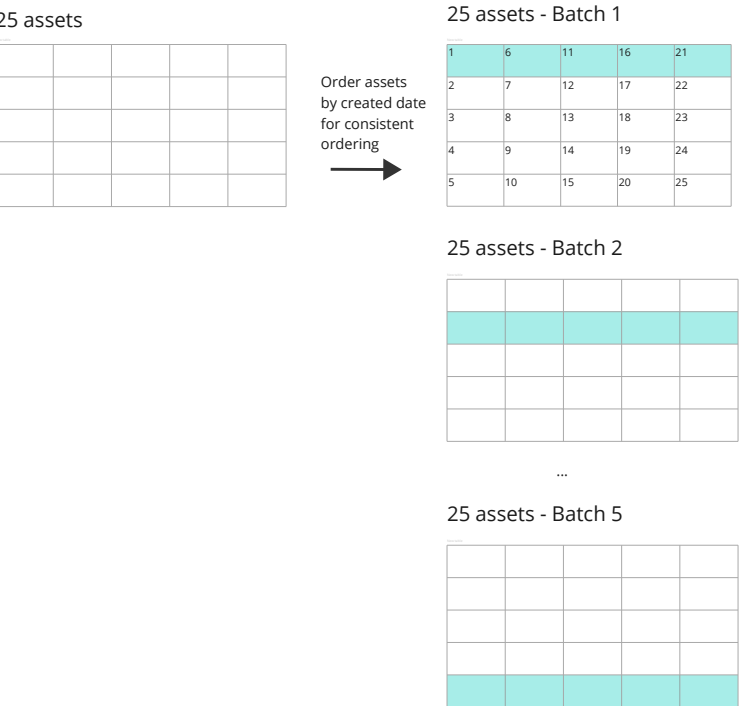


Hackathon → Explore the use of Rust to do our GIS calculations

JB: Think about transparency of what's in the queue if using SQS

Qualification
Cache may
not be fit for
purpose

What we want



What sometimes happens



```
concurrency_factor = event_payload.get('concurrency_factor', 10)
batch = event_payload['batch']

LOGGER.info('Handling Asset Qualification on asset save')
LOGGER.info('Concurrency ordinal: %s', batch)

if switch_is_active('katz_bugfix_robust_debounced_qualification_grouping'):
    asset_qs = FlexAsset.objects.filter(
        qualification_cache__isnull=False, qualification_cache__is_processed=False,
    ).annotate(row=Window(expression=RowNumber(), order_by=F('created').asc()))
else:
    asset_qs = FlexAsset.objects.filter(
        qualification_cache__isnull=False, qualification_cache__is_processed=False,
    ).annotate(row=Window(expression=RowNumber(), order_by=F('modified').asc()))
# This does the initial query for relevant competitions up front to make subsequent queries more
# efficient
relevant_comps = AssetSave.get_relevant_objects()
relevant_comp_qs = Competition.objects.filter(id__in=[comp.id for comp in relevant_comps])
update_list = []
if concurrency_factor > 1:
    subquery_sql, params = asset_qs.query.sql_with_params()
    query = f'select sub.* from ({subquery_sql}) sub where MOD(sub.row, %s)=%s'
    asset_qs = asset_qs.model.objects.raw(query, [*params, concurrency_factor, batch])
```

SQS
Demo

Less
code

Better
scalability

Possibly
degraded
observability

[JWT tokens removed from logs](#)

[Some issues in Sentry don't have a source map](#)

[Developer role doesn't allow users to view lambda information](#)

[Possible to mash the Reset Password button](#)

[Google Meet data analysis](#)

[Race condition between user and role being assigned to user](#)

[Ongoing. Qualification and Visibility competitions not compatible](#)

Admins appearing in non-English

Can't see FlexAssetQualificationCache in ITA admin

[Possible to log into the platform without an organisation role](#)

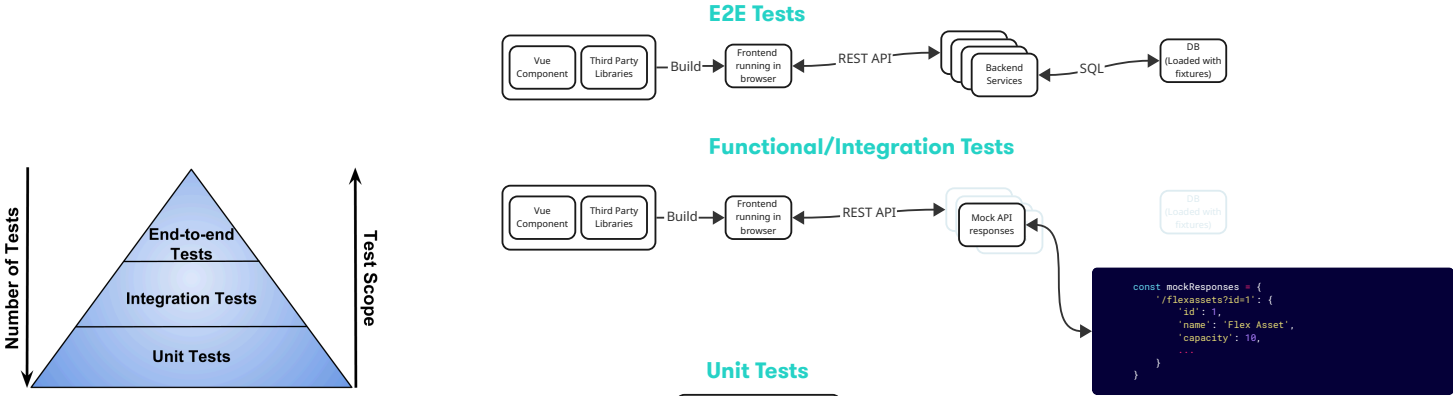
[Possible to log into the platform with a role for an org that no longer exists](#)

[401 from expired token that isn't refreshed when user returns to a tab](#)



Causes downstream noise

Background

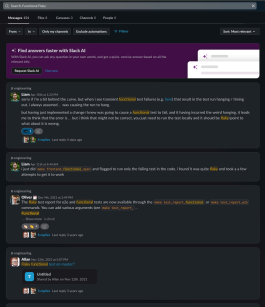


Type	Scope	Speed	Isolation	Use Case Example
End-to-End	Full app	Slow	Low	User flow: Login + Checkout
Integration	Module-to-Module	Medium	Moderate	DB + API working together
Unit	Single unit	Fast	High	Test a math function

Unit Tests	1920	<code>grep -r frontend/src -e "\\$*it(" --include=*.test.js wc -l</code>
Functional Tests	185	<code>grep -r frontend/tests/functional/fixtures -e "\\$*test" wc -l</code>
E2E Tests	87	<code>grep -r frontend/e2e/fixtures -e "\\$*test" wc -l</code>

One observation
- We write far more unit tests could be better suited as integration tests

Motivation



We're on a very old version of TestCafe

Tests that fail as expected, don't report errors in CL so the only solution is to re-run locally

Tests are hanging and then pass on a re-run

There's a lack of developer tooling that may exist in later versions of TestCafe or different tools

A few people have attempted to upgrade but come up against wide-reaching test failures

Online consensus of TestCafe's performance appears to be that it's quite slow in comparison to Playwright

Demo

- 1. Navigate to homepage
- 2. Save token directly to local storage
- 3. Mock out API responses

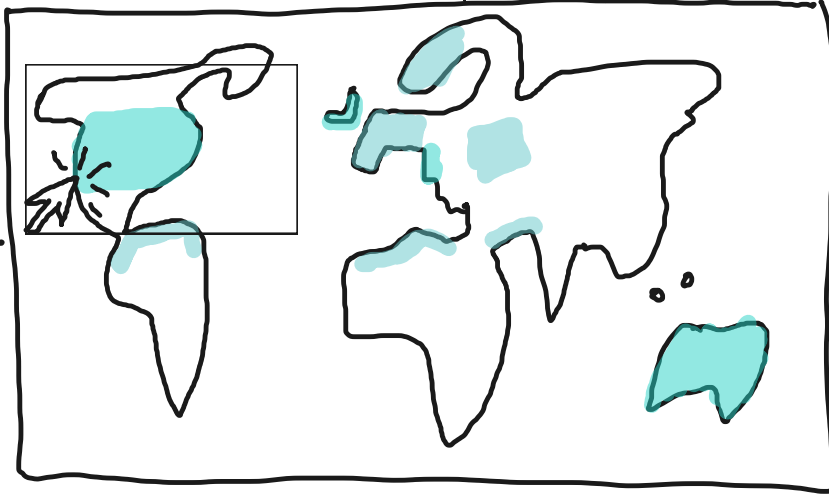
```
test('Can see planned assets when competition selected', async t => {
  await t
    .expect(page.plannedAssetSummary.exists).notOk()
    .click(page.areas.nth(0))
    .expect(page.plannedAssetSummary.exists).ok()
    .expect(page.getAssetPin('storage').exists).ok()
    .expect(page.getAssetPin('non-matching').exists).ok()
    .expect(page.getAssetPin('renewable').exists).ok()
    .hover(page.getAssetPin('storage'))
    .expect(page.isPlannedAssetId('PIAsset1')).ok()
})
```

```
def test_can_see_planned_assets(page_dashboard: Page):
    """Test that planned assets are visible when a competition is selected."""
    planned_assets_summary = page_dashboard.get_by_test_id("planned-assets-summary")
    expect(planned_assets_summary).not_to.be_visible()
    competition_in_sidebar = page_dashboard.get_by_test_id("areas").first
    competition_in_sidebar.click()
    expect(planned_assets_summary).to.be_visible()
    expect(page_dashboard.get_by_test_id("pin-renewable")).to.be_visible()
    expect(page_dashboard.get_by_test_id("pin-non-matching")).to.be_visible()
    expect(page_dashboard.get_by_test_id("pin-storage")).to.be_visible()

    page_dashboard.get_by_test_id("pin-renewable").hover()
    expect(page_dashboard.locator("planned-asset-details span").first).to_have_text(
        "Competition 1 planned asset 1"
    )
```

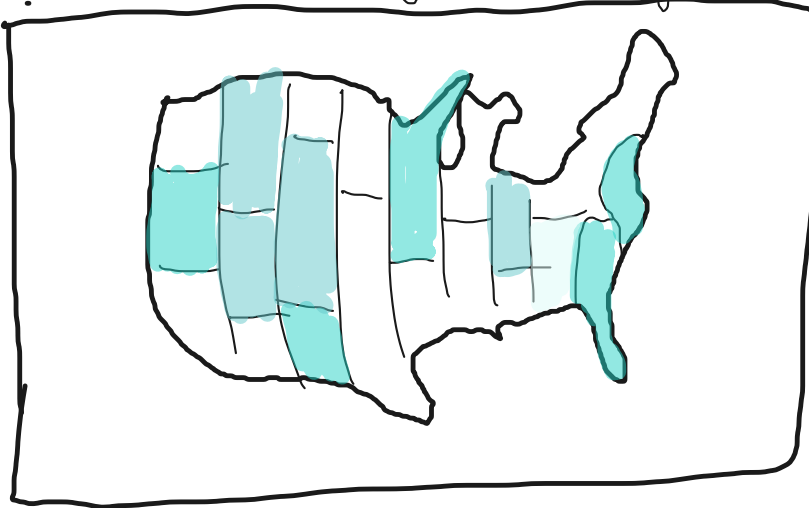
Unrelated Bonus (Django London Meetup)

1. Start with World map Choropleth



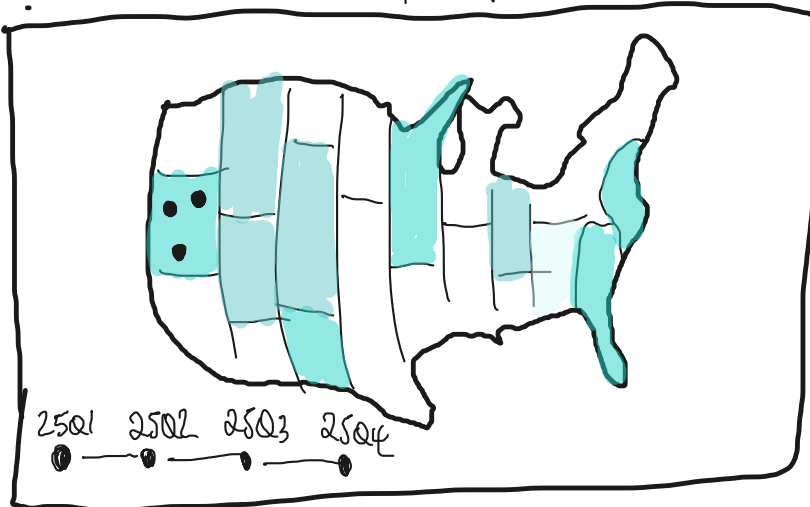
2. Displace regional averages from

various
data series.



intensity
of colour
could be
set by some
high level
figure we
know we
can obtain
for each
country

3. Place pins for specific programs



Asset Type

Storage ☒

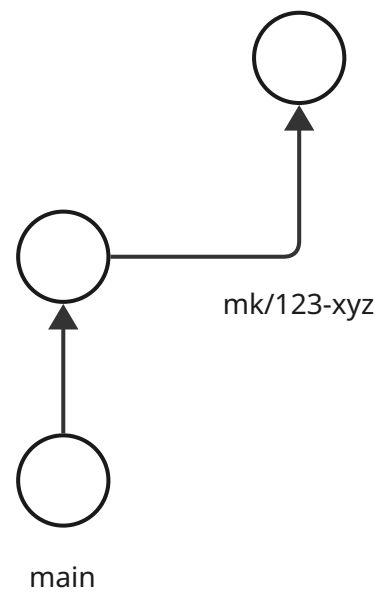
Located in

State

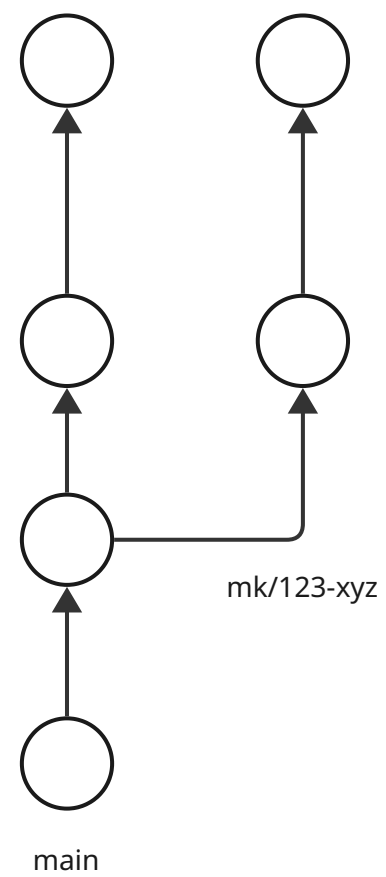
Capacity

2 MW

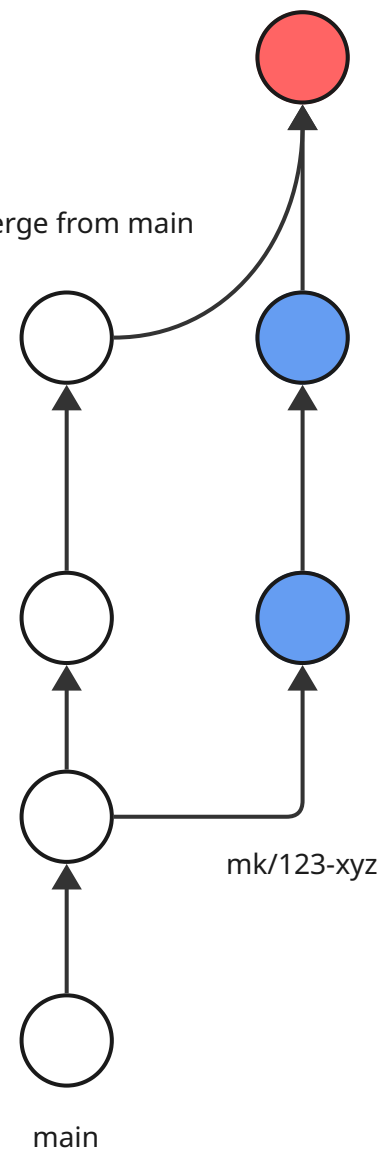
Create Development Branch



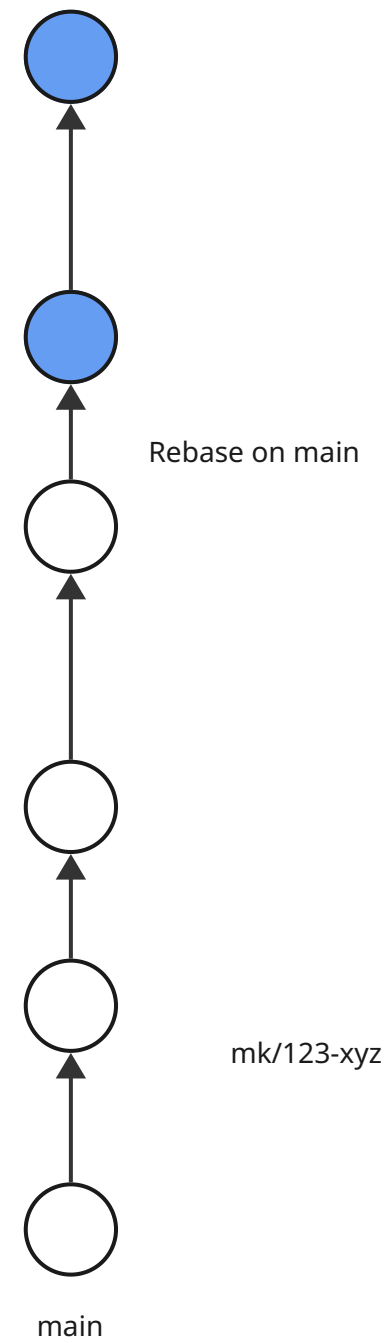
After someone has merged a PR to main

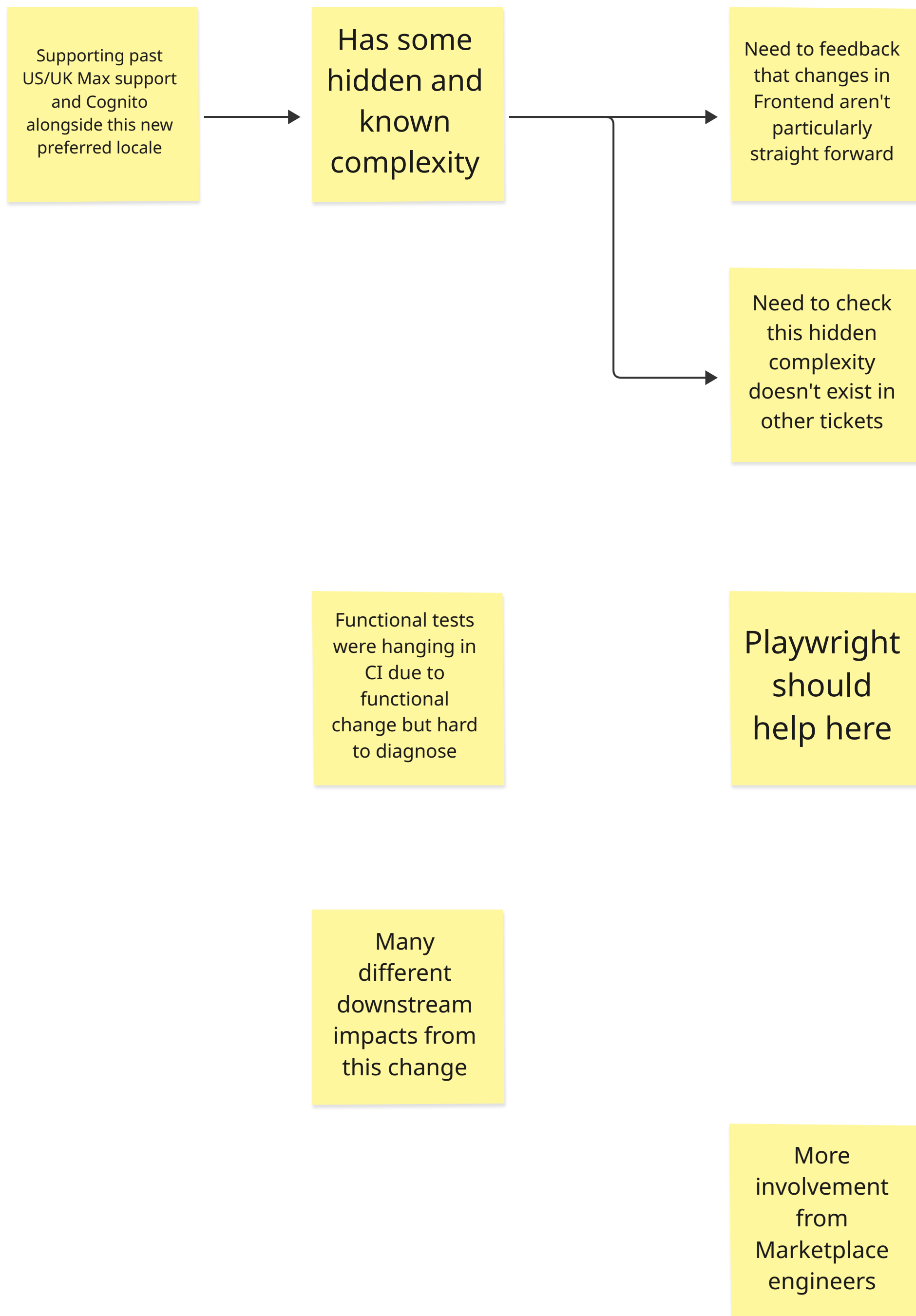


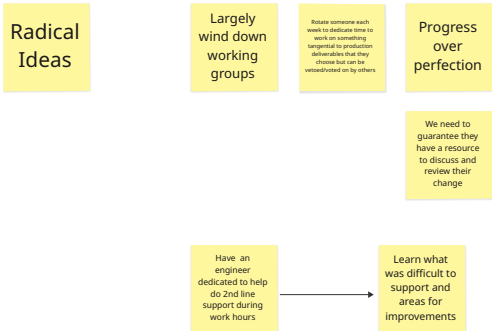
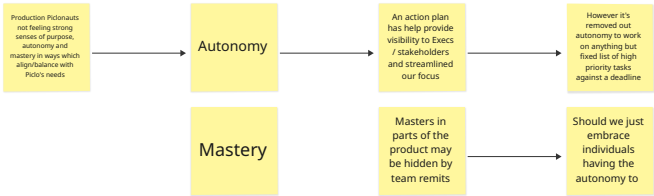
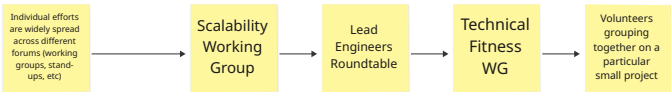
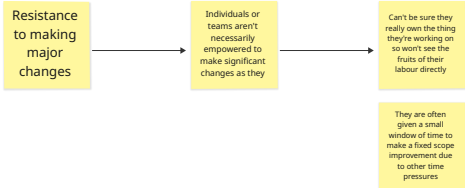
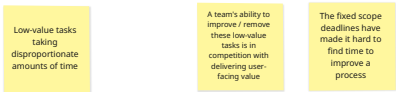
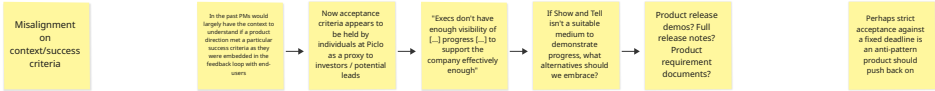
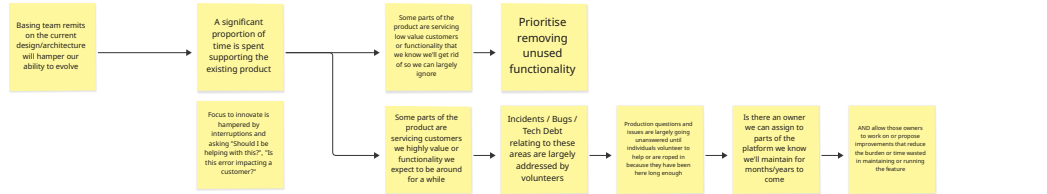
Merge from main



Rebase on main







Andy will give more context in the session, but in short we shouldn't expect things to quickly go back to "normal" (well-defined team remits and long-ish roadmaps to chip away at) — we're still going to be seeking globally-viable product-market-fit. Thank you all for handling the comparative chaos we've had recently and rising to the challenge. But I'm sure we all feel it's possible to organise ourselves better even in this environment, and we'd really appreciate your input into how best to do that. Here are some problems we've identified:

- The product needs to evolve (to meet emerging needs and to simplify), but we don't have a clear picture of the destination
 - There's no quick solution, since it depends where our search for global product-market-fit takes us
 - Basing team remits on the current design/architecture will hamper our ability to evolve
 - Basing team remits on hunches of the future quickly become limiting if/when those hunches prove wrong (this is what we tried in past reteamings)
- We're struggling to maintain cross-org alignment as things change
- We struggle to quickly make high-quality, scalable product changes
 - Resistance to making major changes
 - Tech debt
 - Inflexible architecture
 - Low-value tasks taking disproportionate amounts of time
- Few decisions are being made based on data / user insights
- Misalignment on context/success criteria (both problem-specific and generic/product-wide principles) before starting work, resulting in major changes/rework/waste
- Execs don't have enough visibility of roadmap, progress and solutions to support the company effectively enough
- High risk of competition from AI
 - e.g. moving slower than competitors who embrace AI more fully than us; not moving fast enough when AI threatens our unique value
- Production Piconauts not feeling strong senses of purpose, autonomy and mastery in ways which align/balance with Pico's needs

Suggested workshop agenda:

1. Context recap (10 mins)
2. Problems (30 mins)
 - Do we agree with the problems above?
 - What major problems are missing?
3. Break (10 mins)
4. Principles & solutions (50 mins)

Aiming for solutions which address as many of (or as much 'weight' of) the problems above as possible

Laptop
regularly
crashing

Disk
full

None of my
user accounts
being in
1Password

Half
wound up /
started
WGs

Google
Tasks

Unused
Miro/Google
Docs/Sheets
etc

Several
WIP dev
ideas

Old
Obsidian
kanban
boards

Mitigating
Asset
Topology
issues

What are
the top
monitoring-
uk alerts

No easy way to
see percentage
of released
features in our
team

+1 for
LM
duties

Defunct
Trello
backlogs

Sentry issues
being left
unaddressed

Email
noise

Slack
noise

GitHub	PRs	Notifications	Branches	Discussions	Milestones	 Issues	Projects
Trello	Boards	 Cards	Assignments				
IT	Laptop (8GB)	Laptop (24GB)	Keyboard	Mouse			
Google	 Drive	Docs	Tasks	Mail			
Obsidian	Kanban	Notes					
Slack	Channels	Canvases	Lists	Media	Saved items		
HR	 Deel	 Vitality					
Waffle Features	Create outstanding releases	Question partial releases	Remove unreleased work	Comment on config/partial releases			
Dead code	Milestones service	Prepare templates script	Flaky functional test report	Unused locales	Override assets management command		
Documentation	 Qualification						
Production risks	Observable Framework						
Work adjacent projects	 Focus mode	Production metrics					
Consultation	Questions	Evidence	Timelines				
Hand Over	 File translation PRs						
Random improvements	Avoid error on file uploads in local dev	e321_map box_clusters_legend always OFF	Functional test improvements	Remove template automation switch	Migrate page-access tests to Playwright	Unused step function ARNs	- calculator component
Bets	Monitoring	Nuxt	Playwright	Flex2Max			
Django	 Accounts						

Mon 3rd	Tue 4th	Wed 5th	Thu 6th	Fri 7th	Mon 10th	Tue 11th	Wed 12th
Decide evidence format by 12pm			2026 outlook	Interview		Office	Decision
Personal consultation 3pm							